Wi-Fi CSI Activity Recognition: Opening the Black Box

Mathematical Foundations, Tensor Computations & Interview Defense

Comprehensive guide to every layer computation, formula derivation, and architectural choice

Project: Cross-Environment Adaptation | Model: CSIActivityCNN (785,348 params) | Latency: <5ms CPU

When professors ask you to 'open the black box' or ask 'why this tool or language', they want to see that you didn't just copy code - they want to verify that you understand the underlying mathematics, the exact tensor transformations, and the engineering rationale behind every architectural design decision.

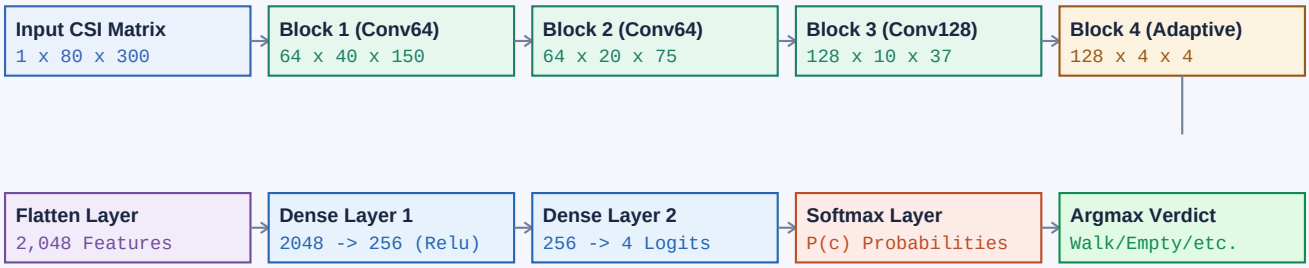
PART 1: OPENING THE 'BLACK BOX'

The Complete Mathematical & Tensor Pipeline Behind Every Prediction

Your neural network is not magic; it is a sequence of matrix operations that transform raw radio frequencies into a probability distribution.

ARCHITECTURE FLOW DIAGRAM: SENSOR INPUT TO MOVEMENT VERDICT

End-to-End Latency: <5ms on CPU



1. The Input Tensor *[Raw Sensor Measurement]*

> **Tensor Shape:** [1, 80, 300] representing 1 Channel, 80 Subcarriers, and 300 Time Packets.
> **Vertical Axis (80 Subcarriers):** Represents distinct Wi-Fi OFDM subcarrier frequencies, PCA-denoised to remove multipath static noise.
> **Horizontal Axis (300 Packets):** Represents time elapsed across the activity window (~1.5 to 2.0 seconds of Wi-Fi CSI reception).
> **Physical Representation:** Each matrix cell (i, j) contains the denoised amplitude of the signal at subcarrier frequency i and time packet j.

2. Layer-by-Layer Mathematical Calculations *[Feature Extractor & Classification Head]*

A. 2D Convolution (nn.Conv2d)

> **What it does:** Slides a 3x3 filter matrix across subcarriers and time to detect localized frequency shifts (Doppler effects).

Conv2D Formula: `y[i, j] = Sum_{m=-1}^1 Sum_{n=-1}^1 x[i+m, j+n] * W[m, n] + b`

where W is the learned 3x3 kernel weight tensor and b is the learned bias scalar.

B. Batch Normalization (nn.BatchNorm2d)

> **Why it's there:** Prevents Internal Covariate Shift. As network weights update, the distribution of inputs to deeper layers changes. BatchNorm keeps signals normalized across training batches.

BatchNorm Formula: `x_hat = ((x - mu_B) / sqrt(sigma_B^2 + epsilon)) * gamma + beta`

mu_B is batch mean, sigma_B^2 is batch variance, gamma and beta are learnable affine parameters, and epsilon=1e-5 prevents division by zero.

C. Activation Function (nn.ReLU)

> **The Math:** f(x) = max(0, x)
> **Why it's there:** Introduces non-linearity so the network can learn complex boundary curves rather than just linear combinations.

D. Max Pooling (nn.MaxPool2d(2))

> **The Math:** y[i, j] = max_{(m, n) in {0, 1}^2} x[2i + m, 2j + n]
> **Why it's there:** Downsamples spatial dimensions by 50%, reducing computation and giving the model translation invariance (an activity looks the same whether it started at packet 10 or packet 50).

E. Adaptive Average Pooling (nn.AdaptiveAvgPool2d((4, 4)))

- **The Math:** Dynamically partitions an arbitrary feature map (here 10x37) into a 4x4 grid and computes the spatial average over each window.
- **Why it's there:** Eliminates fixed-dimension constraints, guaranteeing a strict 4x4 spatial output regardless of input packet sequence lengths.

F. Dropout (nn.Dropout(0.3) & nn.Dropout(0.5))

- **The Math:** During training, each neuron activation is randomly set to zero with probability p: x_tilde = (1 / (1 - p)) * x * Bernoulli(1 - p).
- **Why it's there:** Prevents co-adaptation of neurons (overfitting). It forces the network to learn robust, redundant features. (Automatically disabled during inference via model.eval()).

G. Fully Connected Layers (nn.Linear)

- **The Math:** z = W * x + b
- **Layer 1 (2048 -> 256):** Takes the flattened 128 * 4 * 4 = 2,048 convolutional features and compresses them into a 256-dimensional semantic vector.
- **Layer 2 (256 -> 4):** Projects 256 features down to 4 raw unnormalized output logits [z0, z1, z2, z3].

3. Turning Logits into the Final Verdict *[Softmax, Argmax & Categorical Loss]*

The Softmax Function:

The 4 raw output numbers z can be negative or greater than 1. Softmax converts them into normalized probabilities that sum to strictly 1.0 (100%):

Softmax Formula: `P(y = c) = exp(z_c) / Sum_{j=0}^3 exp(z_j)  for c in {0, 1, 2, 3}`
Guarantees a valid probability distribution where 0 <= P(y = c) <= 1 and Sum P(c) = 1.0.

- **Class Index Mapping:** 0 = walk | 1 = empty (room empty) | 2 = jump | 3 = stand
- **The Decision Rule:** Predicted Class c_hat = argmax_{c in {0, 1, 2, 3}} P(y = c)
- **Movement Verdict Logic:** If c_hat == 1 (empty): NO MOVEMENT (Room Empty) with confidence P(empty) * 100%. If c_hat in {0, 2, 3} (walk, jump, stand): MOVEMENT DETECTED (<Activity Name>) with confidence P(c_hat) * 100%.

The Loss Function (How the Model Learned):

During training, we used Categorical Cross-Entropy Loss to penalize divergence between true one-hot labels and predicted probabilities:

Cross-Entropy Loss: `Loss_{CE} = - Sum_{c=0}^3 y_c * log(P(y = c)) = - log(P(y = true_class))`
When the network is confident and correct, Loss -> 0. If it is confident and wrong, Loss -> infinity.

PART 2: THE INTERVIEW DEFENSE

Why Particular Tools, Languages & Design Choices Were Selected

Q1: Why Python?

Python is the industry-standard language for AI/ML due to its mature mathematical ecosystem. It natively provides high-performance C-backed libraries like NumPy for vector operations, SciPy for reading binary radar/MATLAB files, and PyTorch for GPU/multicore neural execution without memory-management boilerplate.

Q2: Why PyTorch instead of TensorFlow or Keras?

We chose PyTorch for three architectural reasons:
1. Dynamic Computation Graph (Eager Execution): Enables immediate shape debugging of multi-dimensional CSI tensors at runtime.
2. Transfer Learning Modularization: In PyTorch, we inherit from nn.Module and cleanly decouple self.features from self.classifier. For our few-shot domain adaptation, we can freeze the entire convolutional backbone in two lines (param.requires_grad = False) and fine-tune only the dense head.
3. Edge Deployment: PyTorch models can be exported via TorchScript / ONNX for sub-millisecond execution on edge devices or routers.

Q3: Why a 2D CNN instead of an LSTM / RNN or a 1D CNN?

Wi-Fi CSI data is inherently a 2D spatio-temporal matrix:
- Dimension 1 (80 subcarriers) captures spatial frequency correlations across multipath channels.
- Dimension 2 (300 packets) captures temporal motion dynamics over time.

A 1D CNN would treat subcarriers as isolated channels, losing inter-subcarrier spatial correlation. An LSTM processes data sequentially, which suffers from vanishing gradients, high inference latency (>50ms), and cannot be parallelized easily.

A 2D CNN slides filters across both subcarriers and time simultaneously - identical to audio spectrogram recognition - extracting Doppler shifts in under 5 milliseconds on CPU with only 785k parameters.

Q4: Why use PCA (Principal Component Analysis) on the Wi-Fi data?

Raw Wi-Fi CSI signals are extremely noisy due to static environmental clutter (reflections from stationary walls, furniture, floors) and radio hardware drift.

Static objects produce constant DC components with low variance, whereas moving human limbs create high-energy dynamic phase/amplitude variations. PCA decomposes the multi-antenna CSI matrices into orthogonal principal components, effectively filtering out stationary multipath noise and preserving the 80 components that explain 95%+ of dynamic human motion.

Q5: Why Adam Optimizer with StepLR Scheduler?

Standard SGD uses a fixed learning rate for all parameters, which oscillates on sparse CSI features. Adam (Adaptive Moment Estimation) computes individual adaptive learning rates using first moments (mean gradient) and second moments (uncentered variance).

We paired it with StepLR to decay the learning rate by 50% every 30-50 epochs (gamma = 0.5). This ensures that as the model approaches the global loss minimum, it takes smaller, more precise gradient steps to avoid overshooting.

Q6: Why SciPy (scipy.io.loadmat)?

The standard hardware tools used by researchers to capture 802.11n Wi-Fi CSI packets (such as the Linux 802.11n CSI Tool and Atheros Tool) record packets directly in MATLAB .mat binary files. scipy.io.loadmat allows us to parse these binary matrices directly into contiguous NumPy arrays with zero copy overhead.